MacReduce: Federated QLoRA on GPU and Neural Engine Clients

Gautam Diwan
Carnegie Mellon University
gdiwan

Abstract

We fine-tune a language model by treating the processors in a Mac as federated clients, then repeating that grouping across Macs. A client is a GPU, a GPU bound together with the Neural Engine, or those two processors kept separate. Every client trains on a shard of one WikiText-2 mixture, so the heterogeneity is the device. The quantized base stays on the client. The network carries a LoRA adapter delta. Validation perplexity is the quality metric, and adapter bytes are the communication metric. The baseline is FedAvg on one GPU.

1 Introduction

A Mac already contains two trainers that do not share a stack. MLX runs QLoRA on the GPU. Core ML, ANEForge, Orion, and maderix are the routes onto the Neural Engine. A route that does not import, or that never shows residency, is reported as unavailable. It is not replaced by the GPU. When several Macs take part, each machine uses the same grouping, and the adapter has to cross the link between them.

FedAvg cuts that traffic by taking local steps before an average [1]. A longer local run then dominates a plain mean, so unequal step counts use the FedNova rescaling [2]. We label that correction as plain-SGD normalization when the local solver is AdamW. Int8 codes with an error residual shrink the payload. Star, ring, and tree averages agree before quantization and may differ after it.

2 Method

A cohort is an intra-Mac mode and a machine count. GPU mode is one GPU client per Mac. Combined mode is one client per Mac. That client may step on both processors and then emits one delta. Split mode places a GPU client and a Neural Engine client on every Mac, so n Macs yield n GPUs and n Neural Engines. Shards are one seeded partition of the corpus.

Each round starts from the broadcast adapter. FedAvg returns one delta per block of local AdamW steps. The control returns one delta per optimizer step. The mean is a star, a ring, or a tree, with the FedNova rescaling when step counts differ. Quantized clients send int8 codes and a scale, and keep an error residual. Base weights are counted and are not sent.

The GPU path is MLX QLoRA, rank 8, on `mlx-community/Qwen3-0.6B-4bit`. A Neural Engine client runs only after its stack imports and a Qwen3 adapter step exists. One Mac reduces in process. Further Macs join through a relay. This machine is one M4 (Mac16,12).

3 Evaluation

The dataset is Salesforce WikiText-2 raw v1 at revision `b08601e04326c79dfdd32d625aee71d232d685c3`. Training uses the first 4,096 lines for three epochs, batch size 8, and two gradient-accumulation steps. Validation and test stay the full splits. The metric is validation perplexity at each epoch. A finished arm also reports full-split test perplexity, adapter bytes per round, and the base-weight bytes that stayed local.

The baseline is one Mac, GPU only, FedAvg, star aggregation, and an uncompressed delta. The communication control is that cohort with one all-reduce per optimizer step. The other arms cross the three modes with ring and tree aggregation, int8 residuals, and FedNova where local steps differ. A Neural Engine number is kept only with a compute plan or a measured power trace. The comparison fails if the curves agree within run spread, if a Neural Engine client never becomes resident, or if a private stack cannot be pinned. This is an open-ended 10-805 miniproject led by Gautam Diwan (gdiwan). The final report will use the 2024 NeurIPS format. Serving and speculative decoding are later work.

References

[1] B. McMahan et al. Communication-efficient learning of deep networks from decentralized data. *AISTATS*, 2017.

[2] J. Wang et al. Tackling the objective inconsistency problem in heterogeneous federated optimization. *NeurIPS*, 2020.